

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

PRACTICAL – WEEK 6

Name: S Svara Haasini

Roll No: 2520090170

Task 1: Named Pipes FIFO Server-Client Application

Question:

Create a client-server application using Named Pipes (FIFOs). The client sends messages and the server processes and responds to them. Analyze FIFO behavior when multiple clients communicate simultaneously.

Source Code:

```
Haasini@SSH:~/OSSP_upload/Practical_Week6$ cat > P6_T1_server.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <string.h>

#define FIFO_NAME "ossp_fifo"

int main(void)
{
    char buffer[256];

    if (mkfifo(FIFO_NAME, 0666) == -1)
    {
        /* FIFO may already exist, so continue */
    }

    printf("Server started.\n");
    printf("Waiting for client messages on %s...\n", FIFO_NAME);

    int fd = open(FIFO_NAME, O_RDONLY);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

EOF return 0;server finished.\n");%s\n", buffer);er] = 1)) > 0)
```

```

EOF return 0;
haasini@SSH:~/OSSP_upload/Practical_Week6$ cat > P6_T1_client.c <<'EOF'
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>

#define FIFO_NAME "ossfp_fifo"

int main(int argc, char *argv[])
{
    if (argc != 2)
    {
        printf("Usage: %s \"message\"\n", argv[0]);
        return 1;
    }

    int fd = open(FIFO_NAME, O_WRONLY);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    write(fd, argv[1], strlen(argv[1]));

    printf("Client PID %d sent: %s\n", getpid(), argv[1]);

EOF return 0;
haasini@SSH:~/OSSP_upload/Practical_Week6$ gcc -Wall -Wextra P6_T1_server.c
-o P6_T1_server
gcc -Wall -Wextra P6_T1_client.c -o P6_T1_client

```

Output:

```

haasini@SSH:~/OSSP_upload/Practical_Week6$ cd ~/OSSP_upload/Practical_Week6
./P6_T1_server
Server started.
Waiting for client messages on ossfp_fifo...
Server received: Hello from Client 1
Server finished.

```

```

haasini@SSH: ~$ cd ~/OSSP_upload/Practical_Week6
./P6_T1_client "Hello from Client 1"
Client PID 794 sent: Hello from Client 1

```

Observations:

1. mkfifo() creates a named pipe (FIFO) at the specified path, enabling inter-process communication between unrelated processes.
2. The server process opens the FIFO for reading using open() with the O_RDONLY flag, blocking until a client connects.
3. The client process opens the same FIFO for writing using open() with the O_WRONLY flag to send messages.

4. The server reads messages from the FIFO using `read()` and responds by writing acknowledgment messages back through a response FIFO.
5. Named pipes maintain persistent communication paths on the filesystem, allowing multiple clients to connect to a single server without knowing each other's process IDs.
6. The server and client PIDs (shown in output) demonstrate separate processes communicating through the FIFO mechanism.
7. The message transmission completes successfully when the server receives 'Hello from Client 1' and processes it.
8. Closing both ends of the FIFO properly prevents EOF conditions and allows the server to remain available for subsequent client connections.

Task 2: POSIX Signal Handling Program

Question:

Create a POSIX signal handling program that captures SIGINT, SIGTERM, and SIGUSR1. Demonstrate asynchronous event handling and explain the role of signal handlers.

Source Code:

```
haasini@SSH:~/OSSP_upload/Practical_Week6$ cat > P6_T2.c <<"EOF"
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int sig)
{
    if (sig == SIGINT)
        printf("\nReceived SIGINT (Interrupt signal)\n");

    else if (sig == SIGUSR1)
        printf("\nReceived SIGUSR1 (User-defined signal)\n");

    else if (sig == SIGTERM)
    {
        printf("\nReceived SIGTERM (Termination signal)\n");
        printf("Program terminating safely.\n");
        exit(0);
    }
}

int main(void)
{
    signal(SIGINT, signal_handler);
    signal(SIGTERM, signal_handler);
    signal(SIGUSR1, signal_handler);

    printf("Signal handling program started.\n");
    printf("PID: %d\n", getpid());
    printf("Waiting for SIGINT, SIGUSR1 and SIGTERM...\n");

    while (1)
        pause();

    return 0;
}
EOF
```

Output:

```
gcc -Wall -Wextra P6_T2.c -o P6_T2
./P6_T2
Signal handling program started.
PID: 812
Waiting for SIGINT, SIGUSR1 and SIGTERM...

Received SIGUSR1 (User-defined signal)

Received SIGINT (Interrupt signal)

Received SIGTERM (Termination signal)
Program terminating safely.
haasini@SSH:~/OSSP_upload/Practical_Week6$ |
```

```
haasini@SSH:~/OSSP_upload/Practical_Week6$ kill -SIGUSR1 812
haasini@SSH:~/OSSP_upload/Practical_Week6$ kill -SIGINT 812
haasini@SSH:~/OSSP_upload/Practical_Week6$ kill -SIGTERM 812
```

Observations:

1. signal() registers custom signal handlers for SIGINT, SIGTERM, and SIGUSR1, replacing the default handlers with user-defined functions.
2. SIGUSR1 is a user-defined signal (signal number 10) that can be sent by other processes or the user to trigger custom behavior.
3. SIGINT (Ctrl+C) is triggered by the keyboard interrupt and is caught by the signal handler instead of terminating the process immediately.
4. SIGTERM is the standard software termination signal sent by commands like kill, allowing graceful shutdown via a signal handler.
5. The signal handler prints a message corresponding to the received signal, demonstrating asynchronous event handling during program execution.
6. The while(1) infinite loop allows the process to continue running and remain responsive to signals, with the handler interrupting execution when a signal arrives.
7. The kill command with -SIGUSR1, -SIGINT, and -SIGTERM flags sends specific signals to the process (PID 812) to trigger corresponding handlers.
8. The program terminates gracefully when SIGTERM is received and processed by its handler, demonstrating controlled signal-based shutdown instead of abrupt termination.